■■ ScamSentry AI

Technical Requirements Document (TRD)

Technical blueprint for implementing the CyberForge hackathon MVP

1. Purpose

This Technical Requirements Document defines the proposed architecture, components, APIs, data model, security controls, development environment and deployment approach for ScamSentry AI. It is intended to help the two-person team build the project incrementally and integrate frontend and backend without confusion.

2. Proposed System Architecture

The system uses a separated frontend and backend architecture. The React frontend communicates with a FastAPI REST backend. The backend performs validation, normalization, trust scoring, persistence, lookup aggregation and anomaly detection.

```
User → React Frontend → FastAPI REST API → Validation/Trust Services → SQLite Database → Aggregation/Alert Engine
```

3. Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite	Fast single-page web application.
Styling	Tailwind CSS	Rapid responsive UI development.
Backend	Python + FastAPI	REST APIs and automatic API documentation.
Database	SQLite	Fast hackathon setup and local persistence.
ORM	SQLAlchemy	Database models and queries.
Validation	Pydantic	Request/response validation.
API Documentation	FastAPI OpenAPI/Swagger	Endpoint testing and integration.
Version Control	Git + GitHub	Two-person collaboration and version history.

4. Repository Structure

```
ScamSentry AI/ ■■■ backend/ ■ ■■■ app/ ■ ■ ■■■ main.py ■ ■ ■■■ routes/ ■ ■ ■■■ models/ ■ ■ ■■■ services/ ■ ■ ■■■ core/ ■ ■■■ requirements.txt ■ ■■■ .env.example ■■■ frontend/ ■■■ shared/ ■ ■■■ API_CONTRACT.md ■■■ README.md ■■■ .gitignore ■■■ .env.example
```

5. Backend Components

main.py — FastAPI application initialization, route registration and middleware.

routes/ — HTTP endpoint handlers.

models/ — Database entities and schemas.

services/ — Identifier normalization, trust scoring, lookup logic and anomaly detection.

core/ — Configuration, licence state, security helpers and common utilities.

6. Required API Endpoints

Method	Endpoint	Technical Purpose
GET	/health	Return service availability/status.
POST	/reports	Validate and store an anonymous scam report.
GET	/lookup	Normalize and evaluate phone/UPI/URL identifiers.
GET	/map	Return recent area-wise accepted report aggregation.
GET	/alerts	Return unusual activity alerts.
POST	/admin/reset	Reset supported hackathon evaluation state.
GET	/license	Read current licence status.
POST	/license	Update licence dynamically without restart.

7. Data Model

Recommended core entities:

Entity	Key Fields
Identifier	id, type, normalized_value, created_at
ScamReport	id, identifier_id, category, area, description, reporter_fingerprint, status, created_at
Area	id, name, pincode, latitude, longitude
Alert	id, area_id, category, alert_type, severity, anomaly_score, created_at
Licence	id, licence_data/status, expires_at, updated_at

8. Identifier Processing

• **Phone:** Validate Indian phone formats and normalize spacing/prefix differences into a consistent stored representation.

• **UPI:** Validate basic UPI identifier structure and normalize case/whitespace where appropriate.

• **URL:** Validate URL format and normalize safely for duplicate detection.

• The frontend sends raw user input; backend classification and normalization remain authoritative.

9. Trust and Anti-Abuse Engine

The trust engine is a core backend service. It must prevent raw report counts from directly determining whether an identifier is dangerous. Reports should first pass validation and abuse checks.

• Rate limiting to reduce rapid report flooding.

• Reporter deduplication so repeated reports from the same reporter do not count as independent evidence.

• Network/IP-context grouping so many accounts from the same environment are not automatically counted as many independent reporters.

• Duplicate detection using normalized identifier plus report similarity/time-window checks.

• Junk and malformed input rejection.

• Only accepted reports contribute to risk scoring, map aggregation and alerts.

10. Risk and Lookup Logic

Lookup processing should execute in the following sequence: validate input → classify type → normalize value → retrieve accepted reports → compute trusted evidence → calculate risk → return the appropriate verdict.

Supported verdicts: KNOWN_SCAM, SUSPICIOUS, NO_REPORTS and VERIFIED_BUSINESS.

11. Map Aggregation

The map service aggregates accepted reports by area for the selected time window, with seven days as the primary MVP view. The system should return area-level coordinates and counts rather than exposing exact reporter locations.

12. Anomaly Detection and Alerts

SPIKE: Compare accepted reports from the latest 24 hours with the daily average of the previous seven days. Trigger only when the required multiplier and minimum count thresholds are satisfied.

NEW_TYPE: Detect a scam category that appears recently in an area where it was not observed during the configured historical period.

Alerts must be calculated from accepted/trusted reports only.

13. Privacy and Security Requirements

• Never expose reporter identity, contact details, IP addresses or exact reporter locations through public APIs.

• Use area-level aggregation for map and alert output.

• Avoid storing more personally identifying information than required for abuse prevention.

• Use environment variables for secrets and configuration.

• Keep .env files out of Git.

• Use HTTPS for the deployed service.

14. Licence Technical Behaviour

Licence state must be checked dynamically. A new valid licence uploaded through POST /license should restore access without restarting the server. When expired, protected endpoints should return a LICENSE_EXPIRED response while the health and licence management endpoints remain available. Existing application data must remain intact.

15. Frontend Technical Requirements

• Responsive React application optimized for hackathon demonstration.

• Single clear lookup interface for phone numbers, UPI IDs and URLs.

• Fast report flow designed for completion in under 60 seconds.

• Map displaying recent area-wise activity.

• Alert feed showing only unusual activity.

• Consistent handling of API success and error responses.

• Mock-data mode can be used until backend integration is ready.

16. Collaboration Workflow

Mohd Avaish: backend, database, APIs, trust/anti-abuse logic, anomaly detection, integration and deployment.
Sartaj: frontend, UI/UX, lookup/report flows, map/dashboard and visual polish.

• Keep frontend and backend changes separated to reduce merge conflicts.

• Use shared/API_CONTRACT.md as the frontend/backend integration source of truth.

• Integrate early using the real /health and lookup APIs before adding optional features.

17. Development Milestones

1. Foundation: repository structure, environment templates and API contract.

2. Backend online: FastAPI application and GET /health.

3. Core data: database models and persistence.

4. Reporting: POST /reports with validation and anti-abuse checks.

5. Lookup: GET /lookup with normalization and verdict logic.

6. Intelligence: map aggregation and alert detection.

7. Licence: dynamic licence status/update flow.

8. Frontend: integrate all core APIs.

9. Final: test, deploy, rehearse demo and prepare pitch.

This TRD should evolve with implementation decisions. The official hackathon requirements and API/testing specification remain the authoritative reference where exact behaviour is mandated.